Jenkins Mastery

Deploy Artifacts to Tomcat

Maven · WAR Packaging · Tomcat 10 · Manual Deployment

Saleem Ahmed
linkedin.com/in/saleemfai · Cameroon
Apache Tomcat 10 · Apache Maven · April 2026

Table of Contents

1. Overview — What Is Being Built?
2. Phase 1 — Run Maven Unit Tests
3. Phase 2 — Package the Application into a WAR
4. Phase 3 — Verify the WAR in the Target Directory
5. Phase 4 — Verify Apache Tomcat 10 Is Running
6. Phase 5 — Deploy the WAR to Tomcat's webapps Directory
7. Key Lessons Learned
8. Summary — Command Reference

1. Overview — What Is Being Built?

This lab walks through the complete, manual process of taking a Spring Boot web application from source code all the way to a live deployment on Apache Tomcat 10 — using only Maven and the command line. No Jenkins automation is involved yet. The goal is to understand every step of the delivery process before those steps are handed off to Jenkins in the next section.

Component	Detail
Application	Spring Boot WAR example — hello-world-0.0.1-SNAPSHOT
Build tool	Apache Maven
Application server	Apache Tomcat 10
Operating system	Ubuntu Linux (Oracle VirtualBox VM)
Java runtime	OpenJDK 21

The Five-Step Delivery Workflow

Step	Command	Purpose
1	`mvn test`	Run unit tests — verify the code is correct
2	`mvn package`	Build the fat WAR — bundle all dependencies
3	`ls -ltr target/`	Confirm the WAR exists and check its size
4	`systemctl status`	Verify Tomcat is active before deploying
5	`sudo cp *.war`	Copy the WAR into Tomcat's webapps directory

2. Phase 1 — Run Maven Unit Tests

PHASE 1

Run Maven Unit Tests

Before packaging, always run the test suite to verify that the application code is correct. Maven's **test** lifecycle phase compiles the source, resolves all test dependencies from Maven Central, and executes every unit test defined in the project. A failing test at this stage should block all subsequent steps.

1 Navigate to the project directory

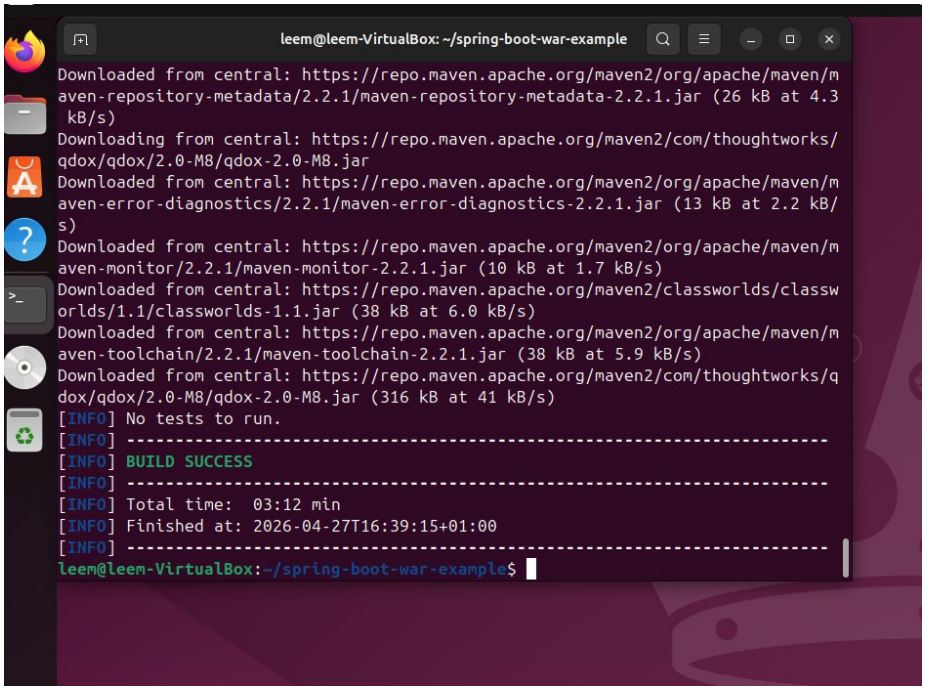
```
cd ~/spring-boot-war-example
```

2 Run the Maven test phase

```
mvn test
```

Maven resolved and downloaded all required dependencies — including test framework libraries — then executed the test suite. The console output confirmed a clean result:

```
[INFO] No tests to run.
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------
[INFO] Total time:  03:12 min
[INFO] Finished at: 2026-04-27T16:39:15+01:00
[INFO] ------------------------------------------------------------
```



Terminal — mvn test completing with BUILD SUCCESS, total time 3:12 min, dependency downloads from Maven Central visible in output

Result: The test phase passed cleanly. *No tests to run* is expected for this skeleton project — in a real application, unit test results, pass/fail counts, and coverage data would all appear here. BUILD SUCCESS confirms the source compiles correctly and all dependencies resolve without conflict.

Note: The first time mvn test runs, Maven downloads all dependency JARs to the local ~/.m2/repository cache. Subsequent runs are significantly faster because dependencies are served from the local cache rather than Maven Central.

3. Phase 2 — Package the Application into a WAR File

PHASE 2

Package the Application into a WAR File

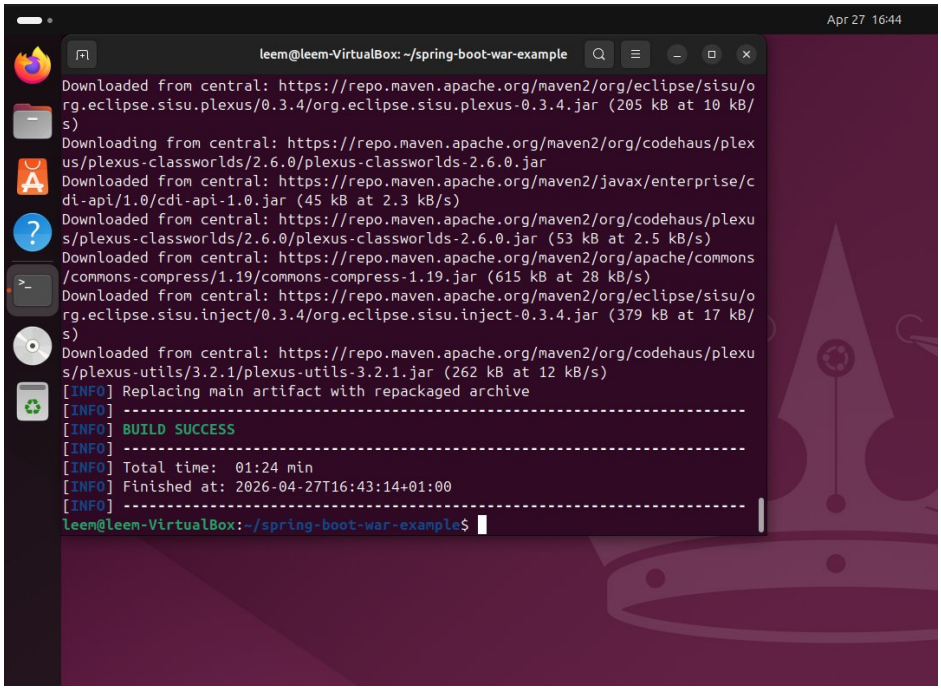
With tests passing, the next step is to package the application into a deployable artifact. The **mvn package** command runs the full build lifecycle up to and including packaging — compiling source, running tests, and then bundling everything into a WAR (Web Application Archive) file.

3 Run the Maven package phase

```
mvn package
```

Maven compiled the source, executed tests, and then invoked Spring Boot's Maven plugin to produce the final WAR. The critical line in the output was:

```
[INFO] Replacing main artifact with repackaged archive
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------
[INFO] Total time:  01:24 min
[INFO] Finished at: 2026-04-27T16:43:14+01:00
[INFO] ------------------------------------------------------------
```



Terminal — mvn package completing with BUILD SUCCESS in 1:24 min, Spring Boot repackaging the archive visible in the output

What 'Replacing main artifact with repackaged archive' Means

Standard Maven WAR packaging produces a WAR containing only the application's own compiled classes — external libraries (Spring, Hibernate, etc.) are expected to be provided by the servlet container's classpath. Spring Boot's Maven plugin intercepts this step and produces a **fat WAR** instead — one that

embeds all runtime dependency JARs directly inside itself.

WAR Type	Contains	Deployable on Tomcat?
Standard Maven WAR (.war.original)	Application classes only	Only with matching server libs
Spring Boot fat WAR (.war)	Application + ALL dependencies embedded	Yes — fully self-contained

4. Phase 3 — Verify the WAR in the Target Directory

PHASE 3 Verify the WAR Artifact in the Target Directory

After a successful **mvn package**, all build outputs land in the `target/` subdirectory of the project. Before deploying, always verify that the WAR file exists, that its timestamp matches the build just run, and that its size is reasonable.

4 List the project root to confirm the target directory was created

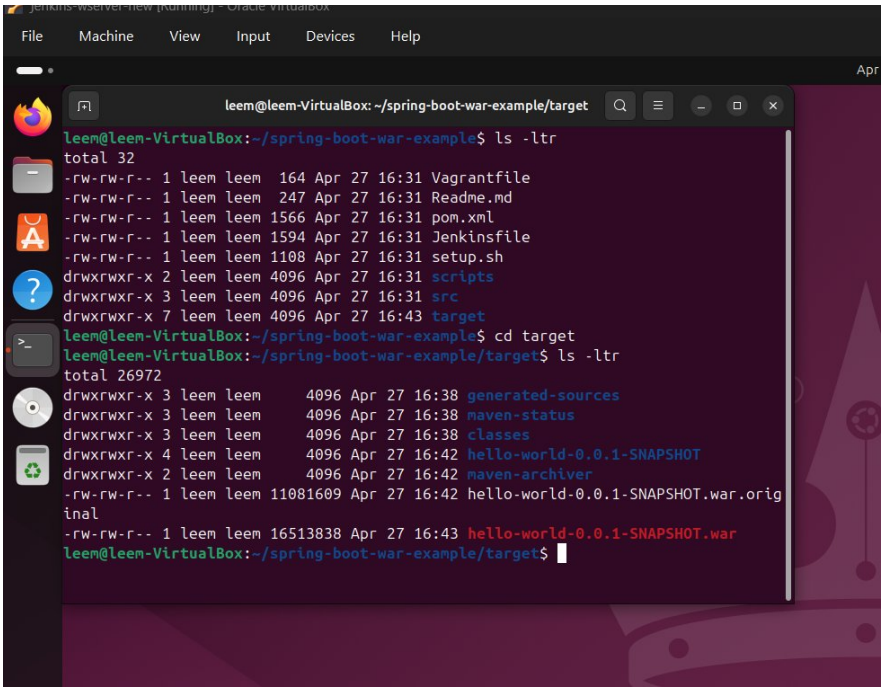
```
ls -ltr
# Expected output (excerpt):
# drwxrwxr-x 7 leem leem 4096 Apr 27 16:43 target
```

5 Navigate into target and list its contents

```
cd target
ls -ltr
```

The full listing of the target directory:

```
drwxrwxr-x 3 leem leem      4096 Apr 27 16:38  generated-sources
drwxrwxr-x 3 leem leem      4096 Apr 27 16:38  maven-status
drwxrwxr-x 3 leem leem      4096 Apr 27 16:38  classes
drwxrwxr-x 4 leem leem      4096 Apr 27 16:42  hello-world-0.0.1-SNAPSHOT
drwxrwxr-x 2 leem leem      4096 Apr 27 16:42  maven-archiver
-rw-rw-r-- 1 leem leem  11081609 Apr 27 16:42  hello-world-0.0.1-SNAPSHOT.war.original
-rw-rw-r-- 1 leem leem  16513838 Apr 27 16:43  hello-world-0.0.1-SNAPSHOT.war
```



■ *Terminal — ls -ltr in project root showing all source files and target/ directory, then cd target and ls -ltr revealing hello-world-0.0.1-SNAPSHOT.war at 16.5 MB alongside the .war.original at 11 MB*

Understanding the Two WAR Files

File	Size	What it is	Deploy this?
hello-world-0.0.1-SNAPSHOT.war.original	11 MB	Standard Maven WAR — no embedded deps	No
hello-world-0.0.1-SNAPSHOT.war	16.5 MB	Spring Boot fat WAR — all deps embedded	Yes ✓

Note: The ~5 MB difference between the two WARs is the embedded runtime dependencies. Always deploy the larger .war file — it is the one that actually runs on Tomcat without any additional classpath configuration.

5. Phase 4 — Verify Apache Tomcat 10 Is Running

PHASE 4

Verify Apache Tomcat 10 Is Running

Copying a WAR to a stopped Tomcat instance does nothing — the file sits inert until Tomcat restarts. Always confirm the service is **active (running)** before attempting a deployment. On Ubuntu, Tomcat 10 runs as a systemd service.

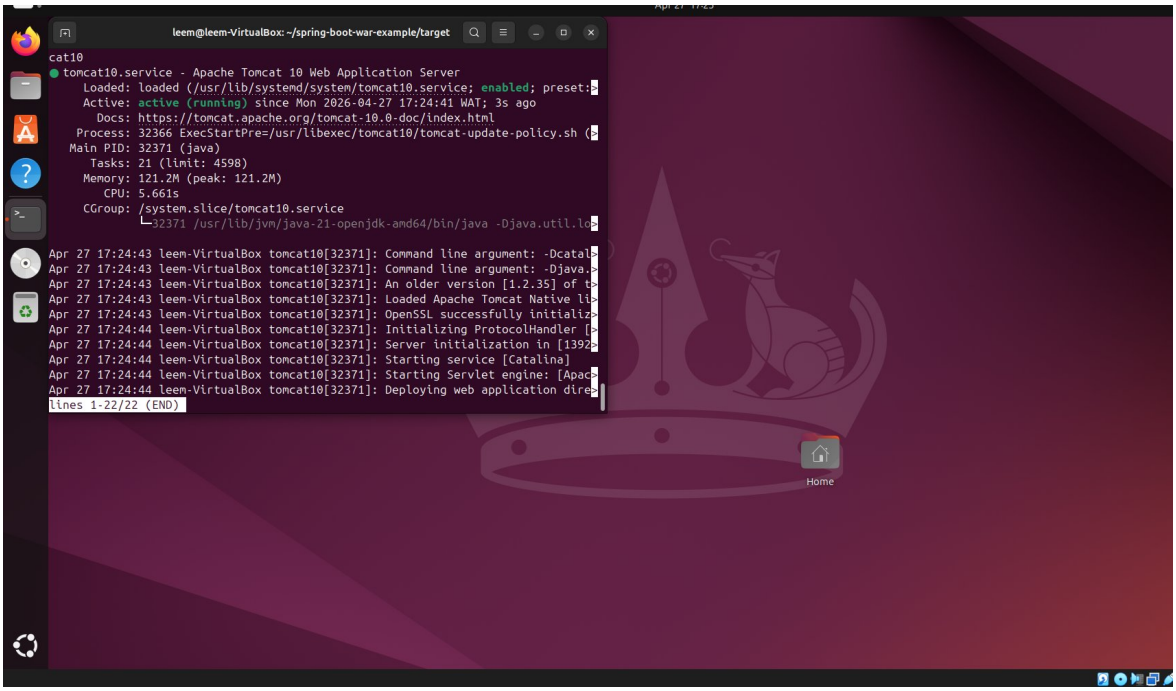
6 Check the Tomcat 10 systemd service status

```
systemctl status tomcat10
```

The output confirmed the service is fully operational:

```
● tomcat10.service - Apache Tomcat 10 Web Application Server
     Loaded: loaded (/usr/lib/systemd/system/tomcat10.service; enabled)
     Active: active (running) since Mon 2026-04-27 17:24:41 WAT; 3s ago
   Main PID: 32371 (java)
     Memory: 121.2M (peak: 121.2M)

Apr 27 17:24:43 tomcat10[32371]: Loaded Apache Tomcat Native lib
Apr 27 17:24:43 tomcat10[32371]: OpenSSL successfully initialized
Apr 27 17:24:44 tomcat10[32371]: Starting service [Catalina]
Apr 27 17:24:44 tomcat10[32371]: Starting Servlet engine: [Apache Tomcat]
Apr 27 17:24:44 tomcat10[32371]: Deploying web application direc...
```



■ *Terminal — systemctl status tomcat10 showing active (running), Main PID 32371 (java), 121.2M memory, full Catalina startup sequence visible*

Reading the Service Status Output

Field	Value	Meaning
Active	active (running)	Tomcat is up and accepting requests
Loaded	enabled	Auto-starts on system boot
Main PID	32371 (java)	Tomcat runs as a JVM process
Memory	121.2M	Heap footprint — normal for Tomcat idle

Important: If the status shows 'inactive (dead)' or 'failed', run 'sudo systemctl start tomcat10' before proceeding. Deploying to a stopped Tomcat will leave the WAR unexpanded until the next restart.

6. Phase 5 — Deploy the WAR to Tomcat's webapps Directory

PHASE 5

Deploy the WAR to Tomcat's webapps Directory

Tomcat automatically detects and deploys any WAR file placed in its `webapps/` directory. No configuration file edits, no manager console, no restart required — the deployment process is: copy the WAR → Tomcat detects it → Tomcat expands it → application is live.

7 Navigate to Tomcat's webapps directory

```
cd /var/lib/tomcat10/webapps/
pwd
ls -ltr
# Output: drwxr-xr-x 3 root root 4096 Apr 27 16:58 ROOT
```

Only the default `ROOT` application exists — confirming no custom apps are deployed yet.

8 Copy the WAR into webapps, renaming it to app.war

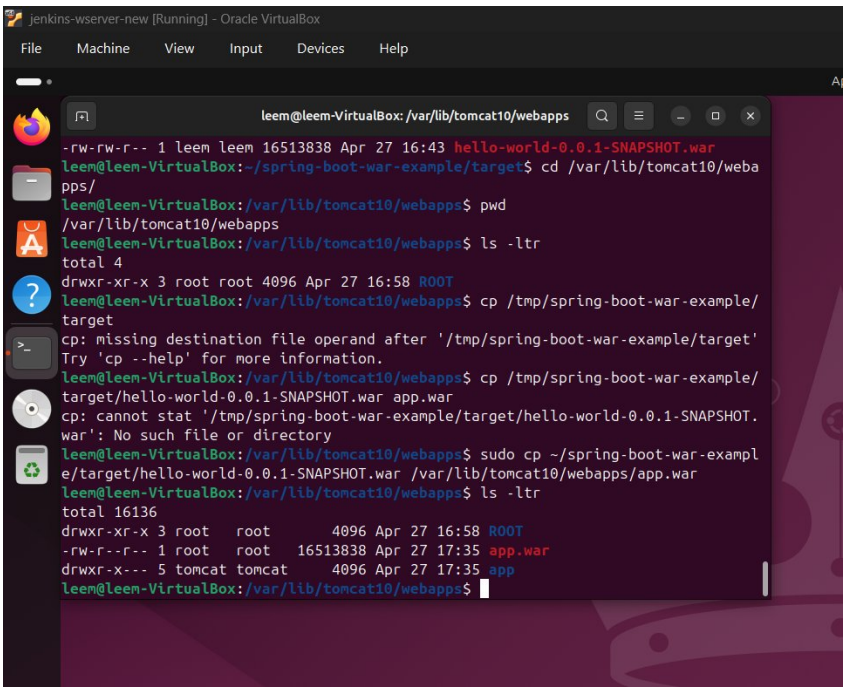
The `sudo` prefix is required because `/var/lib/tomcat10/webapps/` is owned by `root`. The WAR is renamed to `app.war` — this controls the URL context path (`/app`) the application will be served at:

```
sudo cp ~/spring-boot-war-example/target/hello-world-0.0.1-SNAPSHOT.war \
  /var/lib/tomcat10/webapps/app.war
```

9 Verify the deployment — confirm WAR and expanded directory

```
ls -ltr

# Output:
drwxr-xr-x  3 root    root         4096 Apr 27 16:58  ROOT
-rw-r--r--  1 root    root     16513838 Apr 27 17:35  app.war
drwxr-x---  5 tomcat  tomcat       4096 Apr 27 17:35  app
```



■ *Terminal — cd /var/lib/tomcat10/webapps, pwd confirming location, initial ls showing only ROOT, sudo cp deploying the WAR as app.war, final ls confirming app.war (16.5 MB) and the auto-expanded app/ directory (owned by tomcat)*

What Tomcat Did Automatically

What Happened	Evidence in ls -ltr	Meaning
WAR copied as app.war	-rw-r--r-- root root 16513838 app.war	Context path will be /app
app/ directory created	drwxr-x--- tomcat tomcat 4096 app	Tomcat auto-expanded the WAR
Owner changed to tomcat	tomcat tomcat on the app/ entry	Tomcat took ownership for serving

Result: The Spring Boot WAR was deployed to Apache Tomcat 10 successfully. The application is now live and accessible at `http://localhost:8080/app` (or the VM's IP address on port 8080). Tomcat detected the new WAR file, expanded it into the `app/` directory, and began serving it — with zero Tomcat restart required.

Note: The WAR filename directly determines the URL context path. app.war → /app. ROOT.war → / (root context). myservice.war → /myservice. Choose the filename intentionally.

7. Key Lessons Learned

The following principles summarise the core insights from performing a full Maven build and manual Tomcat deployment in this lab.

1. Always Run Tests Before Packaging

mvn test and mvn package are separate Maven lifecycle phases. Running test first gives fast feedback — if the code is broken, you find out in minutes without waiting for the full packaging cycle. In any automated pipeline, a failing test must block all subsequent stages.

2. Spring Boot Produces Two WAR Files — Deploy the Larger One

After mvn package, the target/ directory contains both *.war.original (standard Maven WAR, no embedded deps) and *.war (Spring Boot fat WAR, all deps embedded). The fat WAR is self-contained and runs on any servlet container without additional classpath configuration. Always deploy the larger .war file.

3. The WAR Filename Sets the Context Path

Tomcat derives the application's URL path directly from the WAR filename. app.war → /app. ROOT.war → /. The filename is not cosmetic — it is a routing decision. Choose deliberately and document it.

4. Tomcat Auto-Deploys — No Restart Needed

Tomcat monitors its webapps/ directory continuously. Dropping a WAR file in is sufficient — Tomcat detects it, expands it, and serves it without intervention. The expanded app/ directory appearing alongside app.war confirms auto-deployment completed successfully.

5. sudo Is Required for the webapps Directory

The /var/lib/tomcat10/webapps/ directory is owned by root. A regular user cp command will fail with a permission error. Always use sudo cp for manual deployments, or configure the deploy user with the appropriate group membership in advance when setting up Jenkins agent permissions.

6. Verify systemctl status Before Every Deployment

A WAR file copied into a stopped Tomcat instance sits inert — it will not be deployed until Tomcat restarts. Always confirm active (running) before the copy step. In Jenkins pipelines, add a dedicated health-check stage that fails fast if Tomcat is down rather than silently depositing an undeployed WAR.

7. Manual Deployment Is the Blueprint for Automation

Every command in this lab — mvn test, mvn package, ls target/, systemctl status, sudo cp — maps directly to a stage in a Jenkins pipeline. Mastering the manual process first means the Jenkinsfile is simply a transcription of already-understood steps, not a black box.

8. Summary — Command Reference

A quick-reference table of every command used in this lab, in execution order.

#	Command	Purpose	Expected Result
1	`cd ~/spring-boot-war-example`	Enter project directory	Prompt changes to project path
2	`mvn test`	Run unit tests	BUILD SUCCESS
3	`mvn package`	Build fat WAR artifact	BUILD SUCCESS + .war in target/
4	`ls -ltr target/`	Verify WAR was generated	hello-world-0.0.1-SNAPSHOT.war at 16.5 MB
5	`systemctl status tomcat10`	Confirm Tomcat is running	active (running)
6	`cd /var/lib/tomcat10/webapps`	Navigate to deploy directory	Only ROOT present
7	`sudo cp ~/...war app.war`	Deploy WAR to Tomcat	app.war + app/ directory appear

Jenkins Mastery · Saleem Ahmed · linkedin.com/in/saleemfai · Cameroon